

Lab Project

Digital Safe Box

Digital System Lab

ECE-111

Instructor: Dr Stelios Neophytou

Author's name: Arsenii Risin

School of Sciences and Engineering
Department of Engineering

University of Nicosia

Nicosia, Cyprus

Objective:

We are designing a digital SafeBox system using FPGA. The system stores a password, controls access, and manages a balance of money in BCD format.

Key Components:

- 1) Password Input: Users enter a password by using clk.
- 2) Password Storage: The password is stored in a register as a binary number for efficiency.
- 3) Comparator: The system checks the entered password against the stored password. If correct, it activates an UNLOCK signal.
- 4) Balance Logic: The balance is calculated in BCD adders that handle arithmetic operations with carry between digits.

How it Works:

- 1) The user selects a digit and enters it via switches.
- 2) Each entered digit is stored in the password register.
- 3) When the full password is entered, the comparator checks all digits.
- 4) If the password is correct, the UNLOCK signal allows changes to the balance.
- 5) Users can deposit or withdraw money, and the BCD adders manage the arithmetic with carry propagation.

Part 1. Storage password:

For this part I used components such as 0-9 counter, 0-3 counter, decoder24, 4 and gate and 4 parallel registers

I will attach all my notes about how I created them:

Counters:

Digital System Project

First of all we create a 4-bit counter
which counts from 0 to 7. (0, 1, 2, 3, 4, 5, 6, 7)
Then we create a register that stores 4
numbers of code.
Create digital counter.

$\bar{Q}_4$	$\bar{Q}_3$	$\bar{Q}_2$	$\bar{Q}_1$	$\bar{Q}_0$	$\bar{Q}_4$	$\bar{Q}_3$	$\bar{Q}_2$	$\bar{Q}_1$	$\bar{Q}_0$	$\bar{Q}_4$	$\bar{Q}_3$	$\bar{Q}_2$	$\bar{Q}_1$	$\bar{Q}_0$
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	0	1	0	0	0	1	0	0	0
0	0	0	1	1	0	0	1	1	0	0	1	1	0	0
0	0	1	0	0	0	1	0	0	0	1	0	0	0	1
0	0	1	0	1	0	1	0	1	0	1	0	1	0	1
0	1	0	0	0	1	0	0	0	1	1	0	0	0	1
0	1	0	0	1	1	0	0	1	1	1	0	1	1	1
1	0	0	0	0	1	0	0	0	1	1	0	0	0	1
1	0	0	0	1	1	0	0	1	1	1	0	1	1	1

Flip-flops - inputs

$\bar{Q}_4$	$\bar{Q}_3$	$\bar{Q}_2$	$\bar{Q}_1$	$\bar{Q}_0$
0	0	0	0	0
0	0	0	0	1
0	0	0	1	0
0	0	0	1	1
0	0	1	0	0
0	0	1	0	1
0	0	1	1	0
0	0	1	1	1
0	1	0	0	0
0	1	0	0	1
0	1	0	1	0
0	1	0	1	1
1	0	0	0	0
1	0	0	0	1
1	0	0	1	0
1	0	0	1	1

Register

$\bar{Q}_4$	$\bar{Q}_3$	$\bar{Q}_2$	$\bar{Q}_1$	$\bar{Q}_0$
0	0	0	0	0
0	0	0	0	1
0	0	0	1	0
0	0	0	1	1
0	0	1	0	0
0	0	1	0	1
0	0	1	1	0
0	0	1	1	1
0	1	0	0	0
0	1	0	0	1
0	1	0	1	0
0	1	0	1	1
1	0	0	0	0
1	0	0	0	1
1	0	0	1	0
1	0	0	1	1

(pict1)

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
00	0	0	1	0
01	x	x	x	x
11	x	x	x	x
10	0	0	x	x

$F_2 = Q_1 Q_0$

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
00	0	1	x	x
01	0	1	x	x
11	x	x	x	x
10	x	0	0	x

$F_1 = Q_2 Q_0$

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	x	x	x	x
10	x	x	x	x

$F_3 = Q_2 Q_1 Q_0$

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
00	x	x	x	x
01	x	x	x	x
11	0	0	x	x
10	0	1	x	x

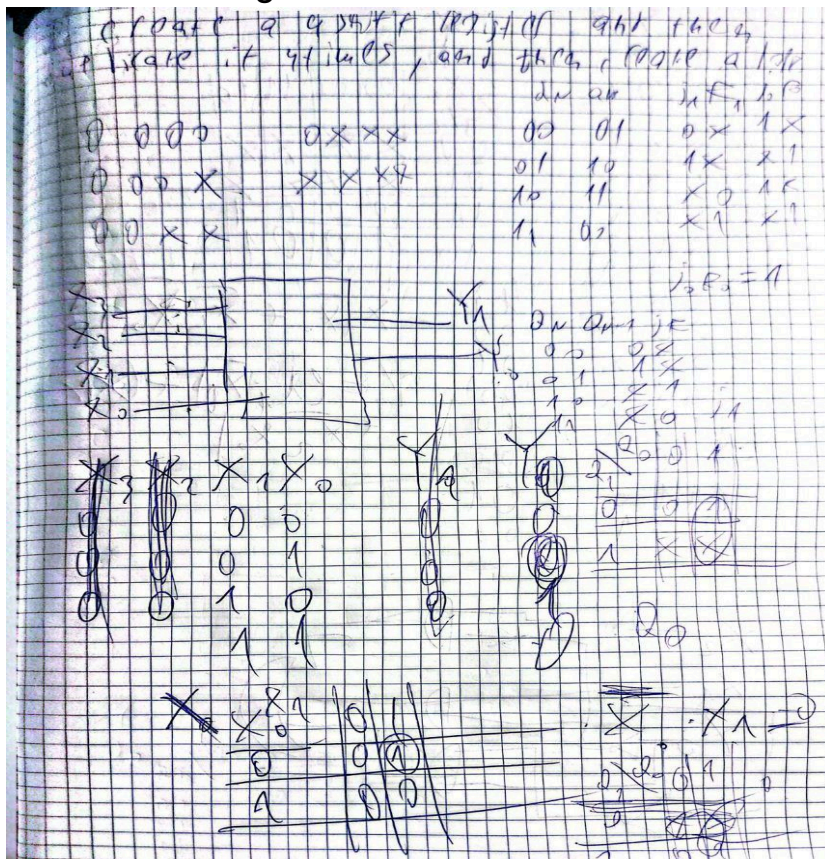
$F_4 = Q_0$

7. Now it'll stored a 241 into shift register

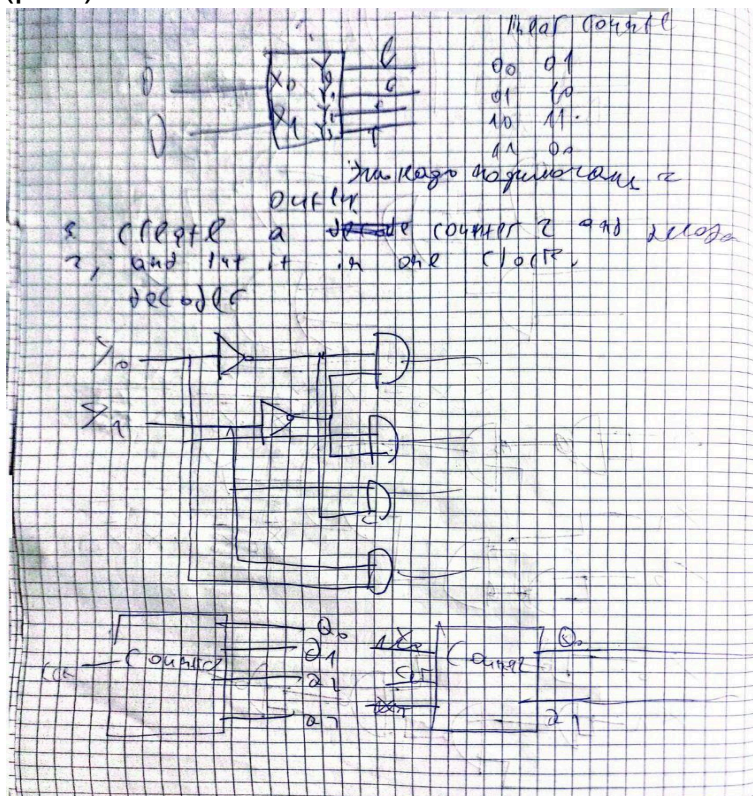
Now it'll create a shift register which associated value and shift left

(pic2)

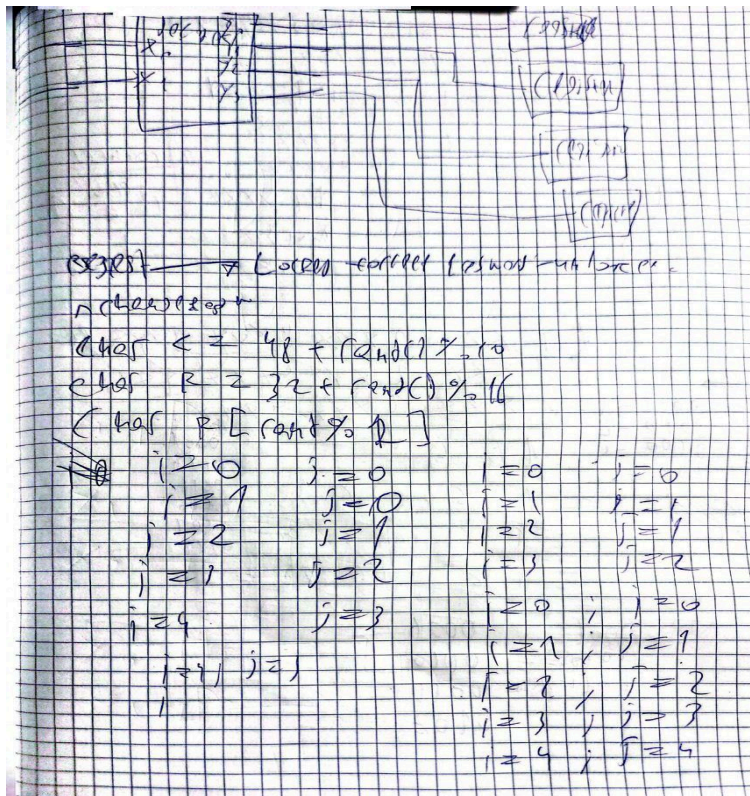
Decoder and logic structure:



(pic3)



(pic4)



(pic5)

Explain how my circuit works. The main inputs are store clk, clk2 and Reset (optional):

clk1 - is needed for the counter to select the word you want to enter.

clk2 - needed to select the desired register

store - needed to save the password

Reset - to delete the password and overwrite it

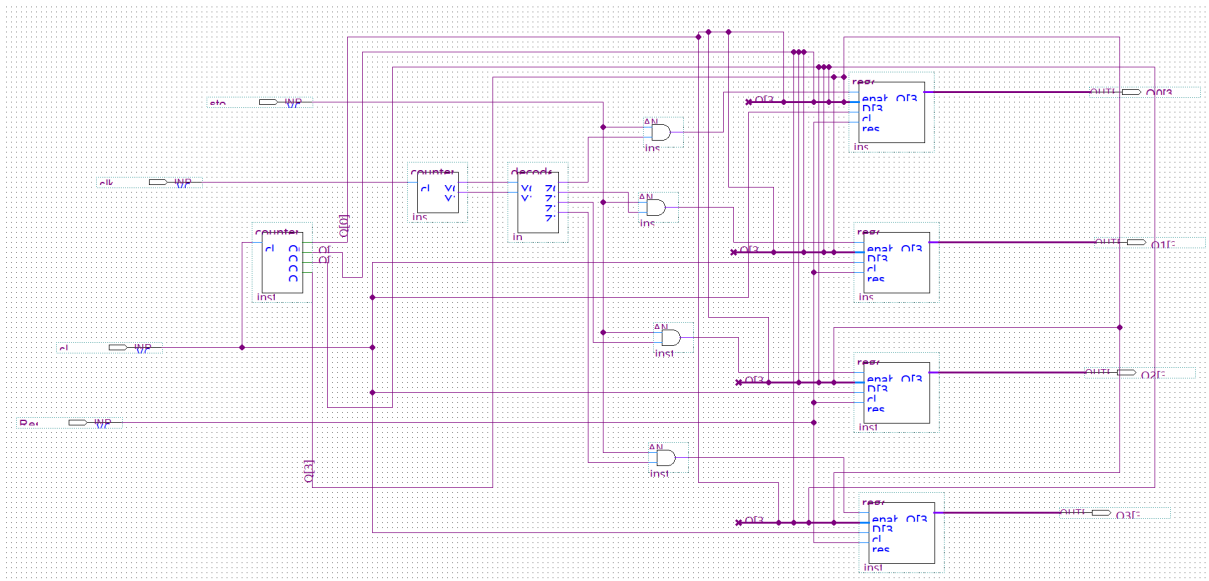
How I built the logic:

1) I simply connected the output of the counter from 0 to 9 to the input of the register (the password has not yet been written down)

2) In order to save a number for each register, I created a 2 and gate, the output of which falls on the enable register. The input of each register is connected to the input to the store and to the output of the 24 decoder.

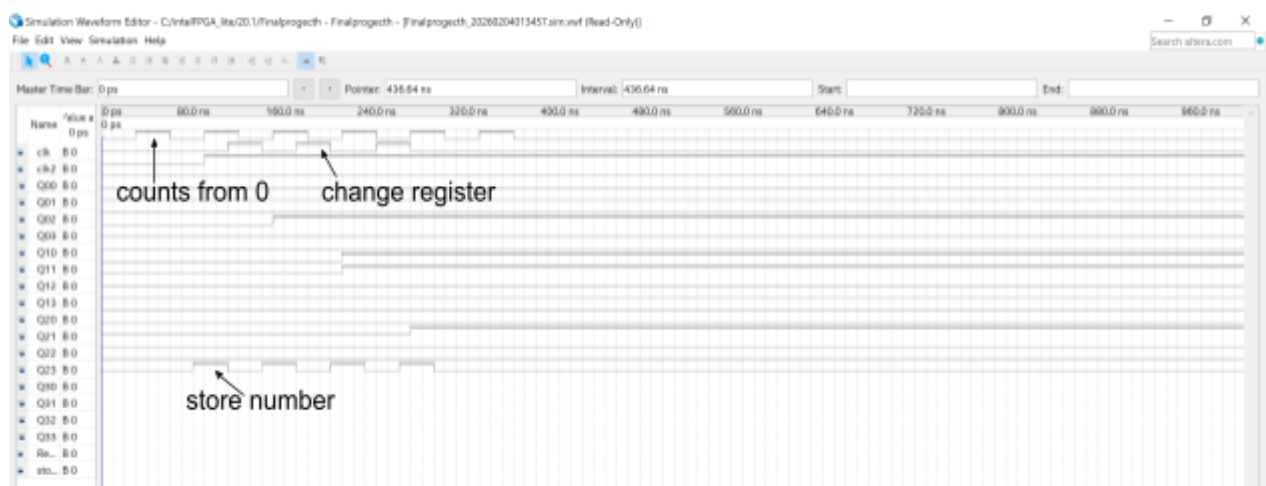
3) Clk2 is responsible for using counter 0-3, the outputs of which are connected to the 24 decoder. Therefore, when the decoder output is 1 and store is 1, then our number is written to the register.

Circuit:

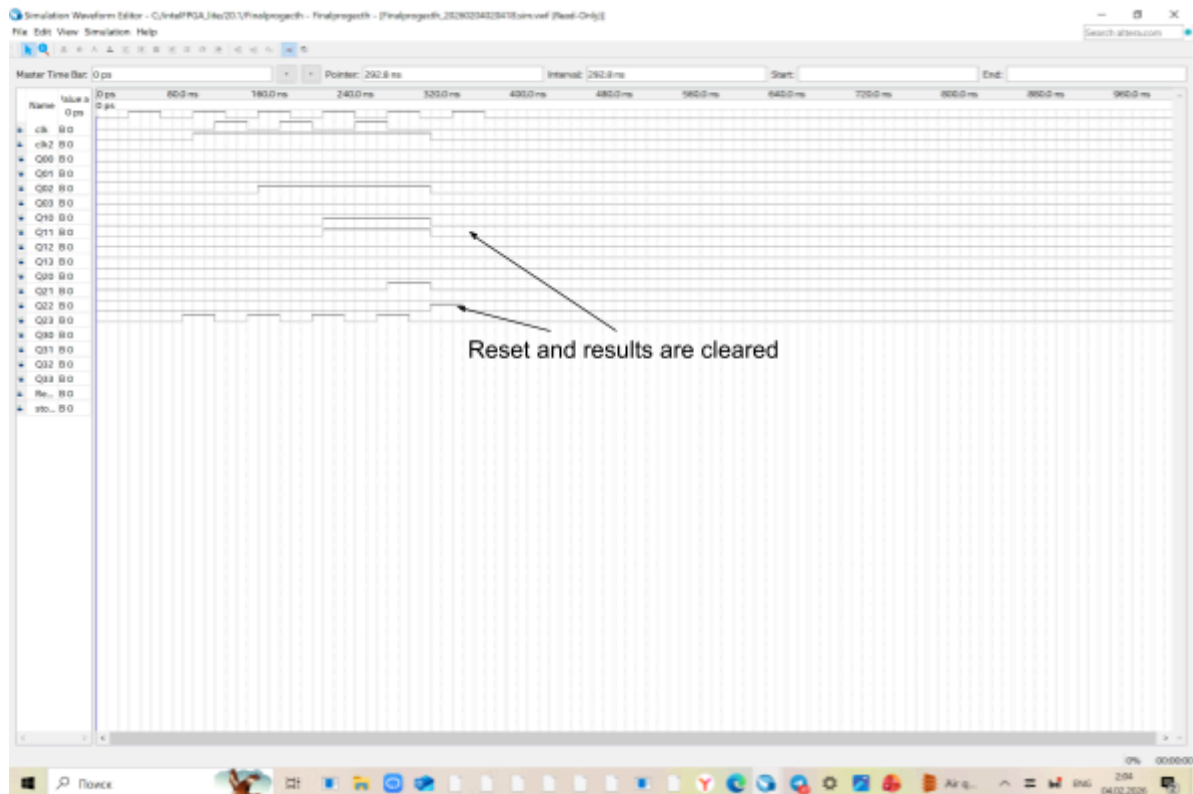


(pict.6)

Now let's simulate this one. Let's say we use the password is: 1234



If we want to clear the password we can use reset:

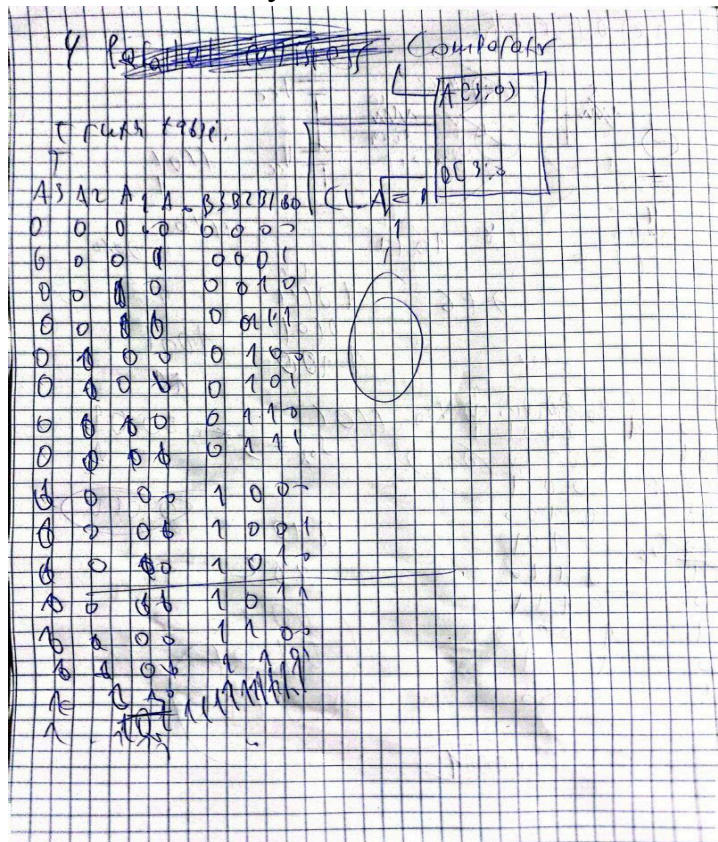


Conclusion:

The password is entered digit by digit using switches and stored in parallel registers. This ensures secure storage and allows the system to remember the correct password for future verification.

Part 2. Check the password:

For this part I used components such as 4- comparators, 4and gate, DFF
I will attach all my notes about how I created them:



(pic7)

Explain how my circuit works. The main inputs are store clk, clk2 and Reset (optional):

clk1, clk2, store, reset - from previous stage

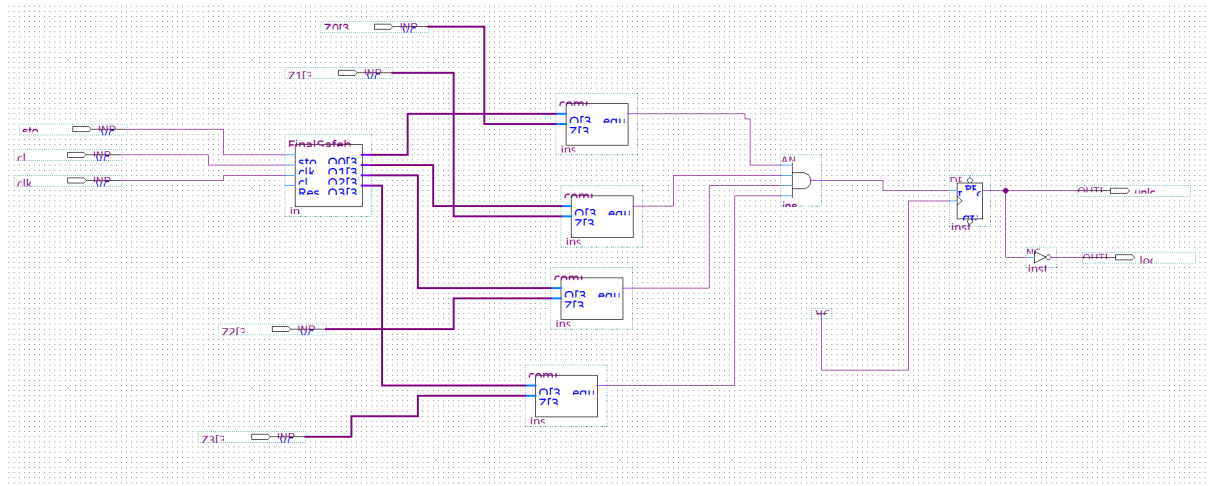
Z0-Z4 - inputs connected to the comparator to check passwords

check - needed to check the password

How I built the logic:

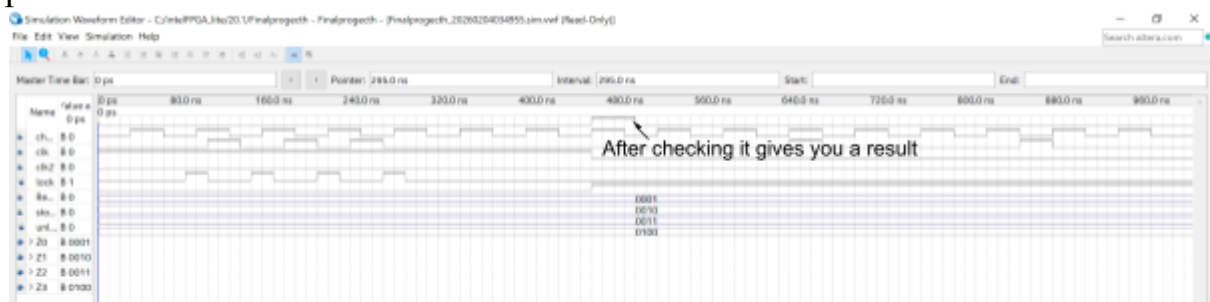
- 1) I connected all the old inputs to the input of store password block that was created in the previous step
- 2) I connected all the outputs from the block to the comparators each of which has two inputs that compare the saved password and the password that you write down
- 3) I connected all the outputs of the comparators into 4and gate if all the results are equal then the output will be 1, if at least one is not equal the output will be 0
- 4) As a result, we connect the output from and gate to DFF, if the value is 0 then the result is lock, if 1 then unlock.

Circuit:



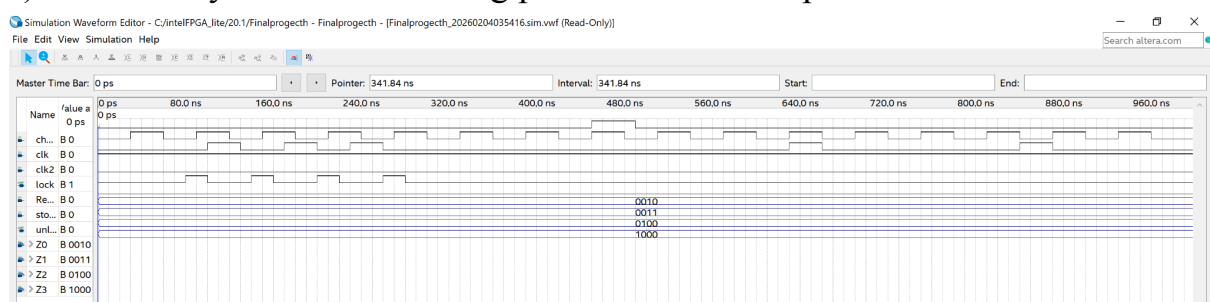
(pict.2)

1) Let's simulate this, let the password be 1234. First, enter the correct password: 1234



Note: you can see when we check this the safebox is open

2) Now let's try to enter the wrong password for example: 2348



Note: the safebox is locked

Conclusion:

Part 3. Balance:

For this part I used components such as BCD 4 bit-adder
I will attach all my notes about how I created them:

Carry In	BCD 1	BCD 2	BCD Sum	Carry Out
0	0000	0000	0000	0
0	0001	0000	0001	0
0	0010	0000	0010	0
0	0011	0000	0011	0
0	0100	0000	0100	0
0	0101	0000	0101	0
0	0110	0000	0110	0
0	0111	0000	0111	0
0	1000	0000	1000	0
0	1001	0000	1001	0
0	1010	0000	1010	0
0	1011	0000	1011	0
0	1100	0000	1100	0
0	1101	0000	1101	0
0	1110	0000	1110	0
0	1111	0000	1111	0
1	0000	0001	0001	0
1	0001	0010	0010	0
1	0010	0011	0011	0
1	0011	0100	0100	0
1	0100	0101	0101	0
1	0101	0110	0110	0
1	0110	0111	0111	0
1	0111	1000	1000	0
1	1000	1001	1001	0
1	1001	1010	1010	0
1	1010	1011	1011	0
1	1011	1100	1100	0
1	1100	1101	1101	0
1	1101	1110	1110	0
1	1110	1111	1111	0
1	1111	0000	0000	1

Carry In	Binary 1	Binary 2	Binary Sum	Carry Out
0	0000	0000	0000	0
0	0001	0000	0001	0
0	0010	0000	0010	0
0	0011	0000	0011	0
0	0100	0000	0100	0
0	0101	0000	0101	0
0	0110	0000	0110	0
0	0111	0000	0111	0
0	1000	0000	1000	0
0	1001	0000	1001	0
0	1010	0000	1010	0
0	1011	0000	1011	0
0	1100	0000	1100	0
0	1101	0000	1101	0
0	1110	0000	1110	0
0	1111	0000	1111	0
1	0000	0001	0001	0
1	0001	0010	0010	0
1	0010	0011	0011	0
1	0011	0100	0100	0
1	0100	0101	0101	0
1	0101	0110	0110	0
1	0110	0111	0111	0
1	0111	1000	1000	0
1	1000	1001	1001	0
1	1001	1010	1010	0
1	1010	1011	1011	0
1	1011	1100	1100	0
1	1100	1101	1101	0
1	1101	1110	1110	0
1	1110	1111	1111	0
1	1111	0000	0000	1

Carry In	Binary 1	Binary 2	Binary Sum	Carry Out
0	0000	1000	1000	0
1	0000	1001	1001	0
1	0000	1010	1010	0

(pic8)

Explain how my circuit works. The main inputs are U[4..0]- UK[4..0], T[4..0]- TK[4..0], H[4..0], HK[4..0], TH[4..0], THK[4..0]

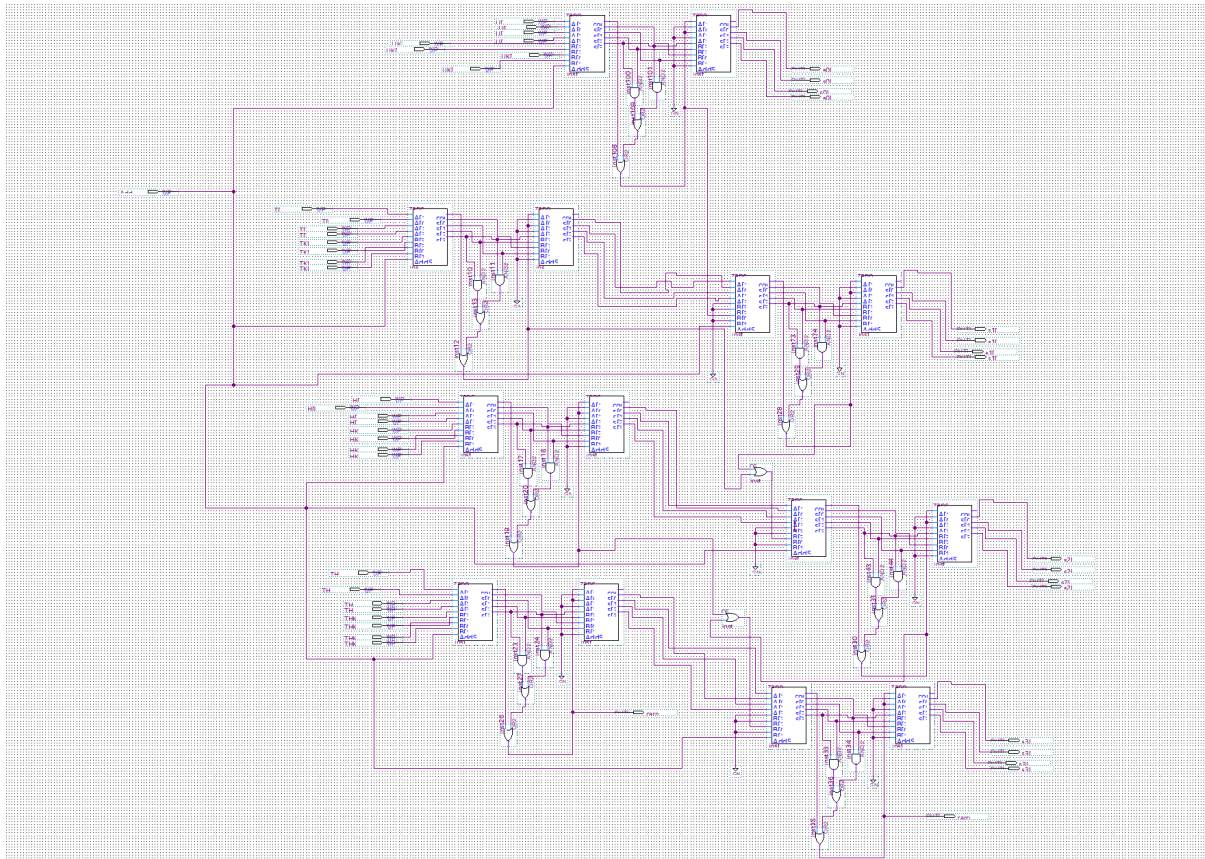
U, T, H, TH - means units, tenths, hundreds and thousands

UK, TK, HK, THK - the same thing but with the addition of the letter K since these inputs are subtraction or addition

How I built the logic:

- 1) Firstly I made a BCD adder so that it would output only decimal digits.
- 2) then I duplicated the BCD for tenths, hundredths and thousandths in order to add this amount with the unit of the previous digit if there is one, thus ensuring the transfer of the unit and making the account valid

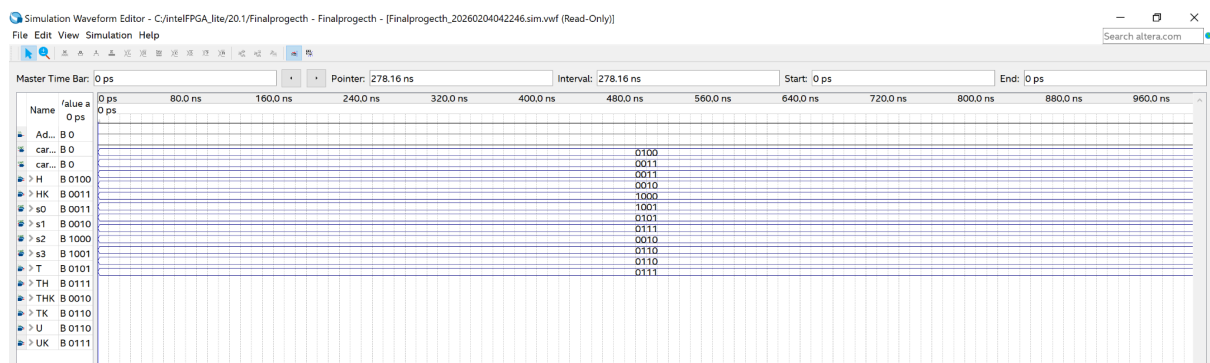
Circuit:



(pic)

Extra note: I couldn't edit the photo properly because I lost my mouse

Now let's stimulate this, let's say we want to put 7456 and add 2367 dollars, the result should be 9823:



Note: You can see that the result of our simulation is **9823** - correct

Conclusion:

The balance is updated using BCD adders. Proper carry handling between digits ensures accurate arithmetic when depositing or withdrawing money.

Analysis:

The SafeBox system was designed using a modular approach, which made the project easier to implement and debug. Separating password storage, password checking, and balance logic allowed each part to be tested independently. Using BCD representation simplified interaction with decimal values and display units, while correct carry propagation ensured accurate arithmetic operations. The project highlights the importance of clear signal organization and proper register usage in FPGA-based digital systems.

Overall Conclusion:

The SafeBox project successfully demonstrates a complete digital system circuit. It combines password storage, password verification, and balance management using BCD logic. The system ensures secure access by comparing stored and entered passwords and allows controlled balance operations only after successful authentication. Proper use of registers, comparators, and BCD adders provides reliable functionality and clear modular design.